Laporan Praktikum

Algortima Dan Struktur Data

Nama : Randy

NIM : 2025573010064

Jurusan Teknologi Informasi Dan Komputer

Prodi Teknik Informatika

Politeknik Negeri Lhokseumawe

2025/2026

Kata Pengantar

Tiada kalimat yang pantas penulis ucapkan kecuali rasa syukur kepada Tuhan Yang Maha Esa atas selesainya laporan praktikum mengenai materi pembelajaran minggu lalu.Tidak lupa pula dukungan baik secara materil dan nonmateril yang diberikan kepada penulis dalam penyusunan makalah ini. Oleh karena itu, izinkan penulis mengucapkan rasa terima kasih kepada:

Dosen Pembimbing : Muhammad Reza Zulman, S.ST., M.Sc

yang telah memberi contoh dan cara dalam mencari bahan buat pembuatan makalah ini sehingga bisa dilaksanakan dengan mudah. Penulis sepenuhnya menyadari karya tulis ini masih jauh dari sempurna, baik dari materi pembahasan maupun tata bahasanya, karena keterbatasan pengetahuan dan kemampuan penulis, untuk itu dengan segala kerendahan hati penulis bersedia menerima kritik dan saran yang bersifat membangun dari semua pihak demi kesempurnaan penulis.

:Lhokseumawe, 28 April 2026

Penulis

Daftar Isi

Bab I 4

Pembahasan 4

1.1 Latar Belakang 4

1.2 Rumusan Masalah 4

1.3 Alat Dan Bahan 4

Bab II 5

Landasan Teori 5

2.1 Fungsi O(1) Konstan 5

2.2 Fungsi O(n) Linear 5

2.3 Fungsi O (n log n) Linearitmik 5

2.4 Fungsi O(n^2) Kuadratik 5

Bab III 6

Prosedur Kerja 6

3.1 Analisi Dan Refactor 6

3.1 Visualisasi Pertumbuhan Big O 9

Bab IV 11

Kesimpulan 11

Bab I

Pembahasan

1.1 Latar Belakang

Big O Notation, yang berfungsi untuk menggambarkan laju pertumbuhan kebutuhan waktu dan memori suatu algoritma terhadap ukuran input. Dengan memahami konsep ini, seorang programmer dapat membandingkan berbagai solusi algoritma dan memilih pendekatan yang paling optimal. Selain itu, analisis kompleksitas juga mencakup dua aspek utama, yaitu Time Complexity yang mengukur waktu eksekusi, serta Space Complexity yang mengukur penggunaan memori.

1.2 Rumusan Masalah

1. Bagaimana cara menganalisis efisiensi suatu algoritma menggunakan Big O Notation?
2. Apa yang dimaksud dengan Time Complexity dan Space Complexity, serta bagaimana cara mengukurnya?
3. Bagaimana mengidentifikasi tingkat kompleksitas algoritma dari pola kode yang digunakan, seperti loop tunggal, nested loop, dan rekursi?
4. Bagaimana cara mengoptimalkan algoritma dari kompleksitas yang tinggi (misalnya $O(n^2)$ atau $O(n^3)$) menjadi lebih efisien (seperti $O(n)$ atau $O(n \log n)$)?

1.3 Alat Dan Bahan

1. Laptop
2. Charger
3. Mouse
4. Modul Analisis Kompleksitas Algortima

Bab II

Landasan Teori

2.1 Fungsi O(1) Konstan

Fungsi dengan kompleksitas O(1) adalah fungsi yang waktu eksekusinya tidak bergantung pada ukuran input. Contohnya adalah fungsi yang hanya melakukan satu operasi sederhana seperti penjumlahan atau pengambilan elemen pertama array.

- Operasi hanya dilakukan satu kali
- Tidak ada loop atau perulangan
- Waktu eksekusi selalu sama berapa pun nilai n

2.2 Fungsi O(n) Linear

Fungsi dengan kompleksitas O(n) melakukan perulangan sebanyak jumlah data.

- Menggunakan satu loop
- Jumlah operasi sebanding dengan nilai n
- Jika n bertambah, waktu juga bertambah secara linear

2.3 Fungsi O (n log n) Linearitmik

Fungsi ini menggabungkan loop linear dengan loop logaritmik.

- Loop luar berjalan n kali
- Loop dalam berjalan $\log_2(n)$ kali
- Total kompleksitas menjadi O(n log n)

2.4 Fungsi $O(n^2)$ Kuadratik

Fungsi ini menggunakan nested loop (loop di dalam loop).

- Dua loop bersarang
- Jumlah operasi = $n \times n$
- Sangat lambat jika n besar

Bab III

Prosedur Kerja

3.1 Analisi Dan Refactor

```
function intersectionSlow(arr1, arr2) {
    const result = [];

    for (let i = 0; i < arr1.length; i++) {
        for (let j = 0; j < arr2.length; j++) {
            if (arr1[i] === arr2[j]) {
                result.push(arr1[i]);
                break;
            }
        }
    }

    return result;
}


function intersectionFast(arr1, arr2) {
    const set = new Set(arr1);
    const result = [];

    for (let num of arr2) {
        if (set.has(num)) {
            result.push(num);
        }
    }

    return result;
}

function groupAnagrams(arr) {

    const map = new Map();

    for (let word of arr) {
        const key = word.split("").sort().join("");

        if (!map.has(key)) {
            map.set(key, []);
        }
```

```
        map.get(key).push(word);
    }

    return Array.from(map.values());
}

function checkSquareSumSlow(arr) {

    const n = arr.length;

    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            for (let k = 0; k < n; k++) {
                if (i !== j && j !== k && i !== k) {
                    if (arr[i] + arr[j] === arr[k] * arr[k]) {
                        return true;
                    }
                }
            }
        }
    }

    return false;
}

function checkSquareSumFast(arr) {

    const sorted = [...arr].sort((a, b) => a - b);
    const n = sorted.length;

    for (let k = 0; k < n; k++) {
        const target = sorted[k] * sorted[k];

        let left = 0;
        let right = n - 1;

        while (left < right) {
            if (left === k) left++;
            if (right === k) right--;

            const sum = sorted[left] + sorted[right];

            if (sum === target) return true;
            if (sum < target) left++;
            else right--;
        }
    }
```

```
    return false;
}

console.log("=== TEST ===");

console.log(intersectionSlow([1, 2, 3], [2, 3, 4]));
console.log(intersectionFast([1, 2, 3], [2, 3, 4]));

console.log(groupAnagrams(['eat', 'tea', 'tan', 'ate', 'nat', 'bat']));

console.log(checkSquareSumSlow([3, 4, 5]));
console.log(checkSquareSumFast([3, 4, 5]));

console.log(checkSquareSumFast([9, 16, 5, 4]));

function generateArray(size) {
    const arr = [];
    for (let i = 0; i < size; i++) {
        arr.push(Math.floor(Math.random() * 1000));
    }
    return arr;
}

const bigArr1 = generateArray(10000);
const bigArr2 = generateArray(10000);
const bigArr3 = generateArray(2000);

console.log("\n=== BENCHMARK FUNGSI A ===");
console.time("Intersection Slow");
intersectionSlow(bigArr1, bigArr2);
console.timeEnd("Intersection Slow");

console.time("Intersection Fast");
intersectionFast(bigArr1, bigArr2);
console.timeEnd("Intersection Fast");

console.log("\n=== BENCHMARK FUNGSI B ===");
const words = Array.from({ length: 10000 }, () =>
    Math.random().toString(36).substring(2, 7)
);

console.time("Group Anagrams");
groupAnagrams(words);
console.timeEnd("Group Anagrams");

console.log("\n=== BENCHMARK FUNGSI C ===");
console.time("Square Sum Slow");
checkSquareSumSlow(bigArr3);
```

```
console.timeEnd("Square Sum Slow");

console.time("Square Sum Fast");
checkSquareSumFast(bigArr3);
console.timeEnd("Square Sum Fast");
```

3.1 Visualisasi Pertumbuhan Big O

```
function fn_O1(n) {

    return n + 1;
}

function fn_On(n) {

    let sum = 0;
    for (let i = 0; i < n; i++) {
        sum += i;
    }
    return sum;
}

function fn_OnLogn(n) {

    let count = 0;

    for (let i = 0; i < n; i++) {
        for (let j = 1; j < n; j *= 2) {
            count++;
        }
    }

    return count;
}

function fn_On2(n) {

    let count = 0;

    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            count++;
        }
    }
```

```
    return count;
}

function benchmarkSemua(ukuranData) {
    for (let n of ukuranData) {
        console.log(`\n=== Ukuran n = ${n} ===`);

        console.time("O(1)");
        fn_O1(n);
        console.timeEnd("O(1)");

        console.time("O(n)");
        fn_On(n);
        console.timeEnd("O(n)");

        console.time("O(n log n)");
        fn_OnLogn(n);
        console.timeEnd("O(n log n)");

        console.time("O(n^2)");
        fn_On2(n);
        console.timeEnd("O(n^2)");
    }
}

benchmarkSemua([100, 500, 1000, 5000, 10000]);
```

Bab IV

Kesimpulan

bahwa algoritma dengan kompleksitas rendah seperti O(1), O(n), dan O(n log n) memiliki performa yang jauh lebih baik dibandingkan algoritma dengan kompleksitas tinggi seperti $O(n^2)$ dan $O(n^3)$. Perbedaan ini semakin terlihat signifikan ketika ukuran data semakin besar, di mana algoritma dengan kompleksitas tinggi mengalami peningkatan waktu eksekusi yang sangat drastis.Selain itu, penggunaan struktur data yang tepat seperti Set dan Map terbukti mampu meningkatkan efisiensi algoritma dengan mengurangi jumlah operasi yang diperlukan. Hal ini menunjukkan adanya konsep time-space trade-off, di mana peningkatan kecepatan sering kali membutuhkan tambahan penggunaan memori.